

## □ 개념 확인

(1) 4장 개념 확인 11번 문제를 해결한 소스코드와 결과를 제시하세요

---

[소스]

```
#include <iostream>
```

```
class Rectangle {
```

```
private:
```

```
    double width;
```

```
    double height;
```

```
public:
```

```
    Rectangle(double width, double height) : width(width), height(height) {}
```

```
// 상수형 함수: 값을 조회만 함 (객체의 멤버 데이터를 변경할 수 없음)
```

```
double getWidth() const {
```

```
    return width;
```

```
}
```

```
double getHeight() const {
```

```
    return height;
```

```
}
```

```
// 비상수형 함수: 멤버 데이터를 수정할 수 있음
```

```
void setWidth(double width) {
```

```
    this->width = width;
```

```
}
```

```
void setHeight(double height) {
```

```
    this->height = height;
```

```
}
```

```
// 상수형 함수: 객체의 상태를 변경하지 않고 넓이를 반환
```

```
double getArea() const {
```

```
    return width * height;
```

```
}
```

```
void Info() const{
```

---

---

```

        std::cout << "===== Rectangle Info ===== " <<std::endl;
        std::cout << "width: " <<width<< std::endl;
        std::cout << "height: " <<height<< std::endl;
        std::cout << "Area: " << getArea() << std::endl;
        std::cout <<std::endl;
    }
};

int main() {
    Rectangle rect(5.0, 3.0);

    std::cout << "Initial >>" << std::endl;
    rect.Info();

    rect.setWidth(10.0);
    std::cout << "width 10 변경 후 >>" << std::endl;
    rect.Info();

    return 0;
}

```

---

[실행결과]

---

(2) 4장 개념 확인 12번 문제 해결을 위한 스코드와 결과를 제시하세요

---

[소스]

```

#include <iostream>

class Point{
private:
    double x;
    double y;

public:
    Point(double x, double y);
    Point();
    ~Point();
    double getX() const;

```

---

---

```
double getY() const;
void setX(double x);
void setY(double y);
void move(const double& dx, const double& dy);
};

// 생성자
Point::Point(double x, double y) : x(x), y(y){ }
Point::Point() : Point(0, 0) {}
Point::~Point(){
    std::cout << "Wnx 좌표 : " << x << std::endl;
    std::cout << "y 좌표 : " << y << std::endl;
    std::cout << "객체가 소멸되었습니다" << std::endl;
}

// 상수형 함수: x 좌표를 반환
double Point::getX() const{
    return x;
}

// 상수형 함수: y 좌표를 반환
double Point::getY() const{
    return y;
}

void Point::setX(double x){
    this->x = x;
}

void Point::setY(double y){
    this->y = y;
}

void Point::move(const double& dx, const double& dy){
    x += dx;
    y += dy;
}

int main()
{
```

---

---

```

// (1, 2)의 좌표를 가진 Point 객체 생성
Point p1{1.0, 2.0};
Point p2{};

std::cout << "---p1 좌표 출력---" << std::endl;
std::cout << "Point 좌표: (" << p1.getX() << ", " << p1.getY() << ")" << std::endl;
// 좌표를 (3, 4)만큼 이동
p1.move(3.0, 4.0);
std::cout << "\n 좌표 이동후:" << std::endl;
std::cout << "Point coordinates: (" << p1.getX() << ", " << p1.getY() << ")" << std::endl;

std::cout << "\n---p2 좌표 출력---" << std::endl;
std::cout << "Point 좌표: (" << p2.getX() << ", " << p2.getY() << ")" << std::endl;
// 좌표를 x=3.4, y=9.3 로 설정
p2.setX(3.4);
p2.setY(9.3);

std::cout << "\n---p2 좌표 재설정 후 좌표 출력---" << std::endl;
std::cout << "Point coordinates: (" << p2.getX() << ", " << p2.getY() << ")" << std::endl;

return 0;
}

```

---

[실행결과]

---

## □ 응용 프로그래밍

(3) 아래의 main()이 실행화면처럼 동작하도록 Person 클래스를 선언하세요. 선언과 구현을 분리합니다

```

int main() {
    Person baby{};
    Person child{"benny", 10};
    cout << "baby name = " << baby.getName() << endl;
    cout << "child name = " << child.getName() << endl;
}

```

|                       |    |             |
|-----------------------|----|-------------|
| 생성자                   | 수행 | Anonymous,0 |
| 생성자                   | 수행 | benny,10    |
| baby name = Anonymous |    |             |
| child name = benny    |    |             |
| 소멸자                   | 수행 | benny       |
| 소멸자                   | 수행 | Anonymous   |

```
>> class Person
```

- 데이터 멤버 : private string name, private int age
- 디폴트 생성자를 제외한 생성자는 생성자 initializer를 사용하여 구현합니다

- 위임 생성자에서 타겟 생성자를 호출하도록 합니다.
- 소멸자
- 접근자 멤버 함수(상수형 함수) : getName(), getAge()

---

[소스]

```
#include <iostream>
using namespace std;

class Person {
    string name;
    int age;
public:
    Person();
    Person(string n, int a);
    ~Person();
    string getName() const;
    int getAge() const;
};

Person::Person() : Person("Anonymous", 0) { }
Person::Person(string n, int a) : name{n}, age{a} {
    cout << "생성자 수행 " << name << "," << age << endl;
}
Person::~~Person() {
    cout << "소멸자 수행 " << name << endl;
}
string Person::getName() const {
    return name;
}
int Person::getAge() const{
    return age;
}

int main() {
    Person baby{};
    Person child{"benny", 10};
    cout << "baby name = " << baby.getName() << endl;
```

---

---

```
    cout << "child name = " << child.getName() << endl;
}
```

---

[실행결과]

---

(4) 다음 main() 함수가 잘 작동하도록 Member 클래스를 작성하고 프로그램을 완성하세요. 클래스 선언과 구현을 분리합니다.

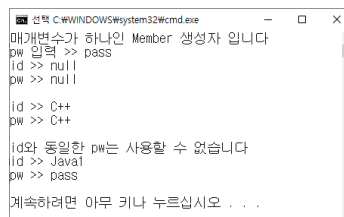
>> Member 클래스

- 멤버 변수 : private string id, private string pw
- 생성자 오버로딩, 생성자 initializer 사용, 위임생성자에서 타겟 생성자 호출
- 소멸자
- 멤버함수

isCheck() : id와 pw를 비교하여 같으면 false, 다르면 true반환, 상수형 함수

display() : 객체 내용 출력, 반환값 없음, 상수형 함수

```
int main() {
    Member mem{};
    Member mem1{"C++", "C++"};
    Member mem2{"Java1"};
    mem.display();
    mem1.display();
    if (mem1.isCheck())
        cout << "사용 가능한 pw 입니다" << endl;
    else
        cout << "id 와 동일한 pw 는 사용할 수 없습니다" << endl;
    mem2.display();
    return 0;
}
```



---

[소스]

```
#include <iostream>
#include <string>
using namespace std;
```

```
//위임 생성자 활용과 생성자 초기화 리스트
```

---

---

```
class Member {
private:
    string id; //아이디
    string pw; //패스워드
public:
    Member();
    Member(string id, string pw);
    Member(string id);
    ~Member(); //소멸자
    bool isCheck() const;
    void display() const;
};

Member::Member() : Member{"null", "null"} {}
Member::Member(string i, string p) : id{ i }, pw{ p } {}
Member::Member(string i) : id{i}{
    cout << "매개변수가 하나인 Member 생성자 입니다" << endl;
    cout << "pw 입력 >> ";
    cin >> pw;
}

Member::~~Member(){
    cout << "Member 객체를 해제 합니다" << endl;
}

bool Member::isCheck() const{
    if (id == pw)
        return false;
    else
        return true;
}

void Member::display() const{
    cout << "id >> " + id << endl;
    cout << "pw >> " + pw << endl << endl;
}

int main() {
```

---

---

```

Member mem{};
Member mem1{ "C++", "C++" };
Member mem2{"Java1"};
mem.display();
mem1.display();
if (mem1.isCheck())
    cout << "사용 가능한 pw 입니다" << endl;
else
    cout << "id 와 동일한 pw 는 사용할 수 없습니다" << endl;
mem2.display();
return 0;
}

```

---

[실행결과]

---

- (5) 제시된 결과처럼 메뉴에서 선택한 연산을 실행하는 프로그램을 작성하세요. 프로그램 종료 시 각 연산을 수행한 횟수를 출력하도록 합니다. +, \*, %, - 연산을 하는 4개의 클래스를 각각 만들고, 각 클래스에는 다음과 같은 멤버를 갖습니다. 제시된 조건 외에 필요한 멤버는 본인이 추가할 수 있습니다

>> 멤버 변수

a(int), b(int);

>>디폴트 생성자

a,b값을 0으로 초기화

>> 멤버 함수

void setValue(int x, int y) : 매개변수를 멤버 변수 값으로 설정

int calculate() 함수 : 연산 결과 반환

```

1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 1
operand >> 4 6
Add : 10
1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 3
operand >> 65 3
Mod : 2
1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 2
operand >> 4 8
Mul : 32
1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 4
operand >> 54 12
Minus : 42
1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 2
operand >> 5 7
Mul : 35
1:(+) , 2:(*) , 3:(%) , 4:(-) 5:exit >> 5
Add = 1
Mod = 1
Mul = 2
Minus = 1

```

---

[소스]

#include <iostream>

---



---

```
using namespace std;
```

```
enum class MENU{PLUS = 1, MUL, MOD, MINUS};
```

```
class Add {  
    int a, b, cnt;  
public:  
    Add() { cnt = 0; }  
    void setValue(int x, int y);  
    int calculate();  
    int getCnt();  
};
```

```
class Mul {  
    int a, b, cnt;  
public:  
    Mul() { cnt = 0; }  
    void setValue(int x, int y);  
    int calculate();  
    int getCnt();  
};
```

```
class Mod {  
    int a, b, cnt;  
public:  
    Mod() { cnt = 0; }  
    void setValue(int x, int y);  
    int calculate();  
    int getCnt();  
};
```

```
class Minus {  
    int a, b, cnt;  
public:  
    Minus() { cnt = 0; }  
    void setValue(int x, int y);  
    int calculate();  
    int getCnt();  
};
```

---

---

```
void Add::setValue(int x, int y) {  
    a = x;  
    b = y;  
}
```

```
int Add::calculate() {  
    ++cnt;  
    return a + b;  
}
```

```
int Add::getCnt() {  
    return cnt;  
}
```

```
void Mod::setValue(int x, int y) {  
    a = x;  
    b = y;  
}
```

```
int Mod::calculate() {  
    ++cnt;  
    return a % b;  
}
```

```
int Mod::getCnt() {  
    return cnt;  
}
```

```
void Mul::setValue(int x, int y) {  
    a = x;  
    b = y;  
}
```

```
int Mul::calculate() {  
    ++cnt;  
    return a * b;  
}
```

```
int Mul::getCnt() {  
    return cnt;  
}
```

```
void Minus::setValue(int x, int y) {
```

---

---

```
    a = x;
    b = y;

}

int Minus::calculate() {
    ++cnt;
    return a - b;
}

int Minus::getCnt() {
    return cnt;
}

int menu() {
    int m;
    cout << "1:(+) , 2:(*), 3:(%), 4:(-) 5:exit >> ";
    cin >> m;
    return m;
}

int main() {
    int a, b;
    Add add;
    Mod mod;
    Mul mul;
    Minus minus;

    while (true) {
        int choice = menu();
        if (choice == 5)
            break;

        cout << "operand >> ";
        cin >> a >> b;

        switch (static_cast<MENU>(choice)){
            case MENU::PLUS :
                add.setValue(a, b);
                cout << "Add : " << add.calculate()<<endl;
                break;
            case MENU::MUL:
                mul.setValue(a, b);
                cout << "Mul : " << mul.calculate() << endl;
```

---

---

```

        break;
    case MENU::MOD:
        mod.setValue(a, b);
        cout << "Mod : " << mod.calculate() << endl;
        break;
    case MENU::MINUS:
        minus.setValue(a, b);
        cout << "Minus : " << minus.calculate() << endl;
        break;
    }
}

cout << "Add = " << add.getCnt() << endl;
cout << "Mod = " << mod.getCnt() << endl;
cout << "Mul = " << mul.getCnt() << endl;
cout << "Minus = " << minus.getCnt() << endl;
cout << "프로그램 종료" << endl;
return 0;
}

```

---

[실행결과]

---

- (6) main()함수와 실행 결과를 참고하여 CoffeeMachine 클래스를 작성하고 프로그램을 완성하세요. 단, 커피와 물, 설탕 최대 용량은 10으로 합니다.

```

커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 2
커피 머신 상태, 커피:9 물:8 설탕:10
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 1
커피 머신 상태, 커피:8 물:7 설탕:10
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 3
커피 머신 상태, 커피:7 물:5 설탕:8
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 3
커피 머신 상태, 커피:6 물:3 설탕:6
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 5
water를 7리터 보충
커피 머신 상태, 커피:6 물:10 설탕:6
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 2
커피 머신 상태, 커피:5 물:8 설탕:6
커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> 4
커피 머신을 종료합니다

```

```

int main() {
    CoffeeMachine java{10, 10, 10}; // 커피량:10, 물량:10, 설탕:10 으로 초기화
    java.drinkEspresso(); // 커피 1, 물 1 소비
    java.drinkAmericano(); // 커피 1, 물 2 소비
    java.drinkSugarCoffee(); // 커피 1, 물 2, 설탕 2 소비
    java.show(); // 현재 커피 머신의 상태 출력
}

```

```
    java.fill(); // 현재 머신에 남아 있는 커피와 물, 설탕 수량이 5미만 이면 최대 용량으로 채워 넣기
}
```

---

[소스]

```
#include <iostream>

using namespace std;

enum class MENU{
    Espresso = 1,
    Americano=2,
    SugarCoffee=3,
    Fill=5
};

class CoffeeMachine{
    int coffee, water, sugar; // 전체 커피, 물량, 설탕 량
public:
    CoffeeMachine(int c, int w, int s);
    void drinkEspresso();    // 커피 1, 물 1 소비
    void drinkAmericano();   // 커피 1, 물 2 소비
    void drinkSugarCoffee(); // 커피 1, 물 2, 설탕 1 소비
    void fill();             // 커피 10, 물 10, 설탕 10 으로 채우기
    void show() const;       // 잔량 출력
};

CoffeeMachine::CoffeeMachine(int coffee, int water, int sugar) : coffee{coffee}, water{water},
sugar{sugar}{}

void CoffeeMachine::drinkEspresso(){ // 커피 1, 물 1 소비
    if (coffee < 1 || water < 1)
        return;
    // 커피 1, 물 1 소비
    coffee--;
    water--;
}

void CoffeeMachine::drinkAmericano(){ // 커피 1, 물 2 소비
    if (coffee < 1 || water < 2)
        return;
```

---

---

```
// 커피 1, 물 2 소비
coffee--;
water -= 2;
}

void CoffeeMachine::drinkSugarCoffee(){ // 커피 1, 물 2, 설탕 1 소비
    if (coffee < 1 || water < 2 || sugar < 1)
        return;
    // 커피 1, 물 2, 설탕 1 소비
    coffee--;
    water -= 2;
    sugar-=2;
}

void CoffeeMachine::fill(){ // 커피, 물, 설탕 전체 용량에서 부족한 만큼 채워넣기
    if (coffee < 5) {
        cout<<"coffee 를 "<<10-coffee<<"g 보충"<<endl;
        coffee += (10 - coffee);
    }
    if (water < 5) {
        cout<<"water 를 "<<10-water<<"리터 보충"<<endl;
        water += (10 - water);
    }
    if (sugar < 5) {
        cout<<"sugar 를 "<<10-sugar<<"g 보충"<<endl;
        sugar += (10 - sugar);
    }
}

void CoffeeMachine::show() const{ // 잔량 출력
    cout << "커피 머신 상태, 커피:" << coffee << "Wt";
    cout << "물:" << water << "Wt";
    cout << "설탕:" << sugar << "Wt" << endl;
}

int main(){
    int menu;
    CoffeeMachine java(10, 10, 10); // 커피량:10, 물량:10, 설탕:10 으로 초기화

    while (true) {
        cout << "커피 종류 선택 : Espresso = 1, Americano=2, SugarCoffee=3, Exit=4, Fill=5 >> ";
```

---

---

```
cin >> menu;

if (menu == 4) {
    break;
}

switch (static_cast<MENU>(menu)) {
case MENU::Espresso:
    java.drinkEspresso(); // 커피 1, 물 1 소비
    break;
case MENU::Americano:
    java.drinkAmericano(); // 커피 1, 물 2 소비
    break;
case MENU::SugarCoffee:
    java.drinkSugarCoffee(); // 커피 1, 물 2, 설탕 1 소비
    break;
case MENU::Fill:
    java.fill();
    break;
default:
    cout << "메뉴 없음" << endl;
}
java.show();
}
cout << "커피 머신을 종료합니다" << endl;
return 0;
}
```

---

[실행결과]

---

- (7) 제시된 main() 함수를 참고하여 실행 결과와 같이 동작하도록 계좌 관리를 위한 클래스 Account를 구현하세요. 단, 클래스는 헤더파일과 cpp 파일로 분리하여 작성합니다. 열거형 클래스를 사용하지 않고 해결하여도 됩니다.

```

-----
menu : 1. 입금, 2. 출금 3. 조회 4. 종료 >> 1
입금액 >> 30000
C++의 입금 액은 30000
C++의 잔액은 30000
-----
menu : 1. 입금, 2. 출금 3. 조회 4. 종료 >> 2
출금액 >> 10000
C++의 출금 액은 10000
C++의 잔액은 20000
-----
menu : 1. 입금, 2. 출금 3. 조회 4. 종료 >> 3
C++의 잔액은 20000
-----
menu : 1. 입금, 2. 출금 3. 조회 4. 종료 >> 4
C++: 객체 소멸

```

```
#include <iostream>
```

```
#include "account.h"
```

```
using namespace std;
```

```
int main() {
```

```
    Account a{"C++", 0};
```

```
    MENU choice;
```

```
    int menu, money;
```

```
    do {
```

```
        cout << "-----" << endl;
```

```
        cout << "menu : 1. 입금, 2. 출금 3. 조회 4. 종료 >> ";
```

```
        cin >> menu;
```

```
        choice = static_cast<MENU>(menu);
```

```
        if(choice == MENU::QUIT)
```

```
            break;
```

```
        switch (choice) {
```

```
        case MENU::DEPOSIT:
```

```
            cout << "입금액 >> ";
```

```
            cin >> money;
```

```
            a.deposit(money);
```

```
            cout << a.getOwner() << "의 입금 액은 " << money << endl;
```

```
            cout << a.getOwner() << "의 잔액은 " << a.check() << endl;
```

```
            break;
```

```
        case MENU::WITHDRAW:
```

```
            cout << "출금액 >> ";
```

```
            cin >> money;
```

```
            cout << a.getOwner() << "의 출금 액은 " << a.withdraw(money) << endl;
```

```
            cout << a.getOwner() << "의 잔액은 " << a.check() << endl;
```

```
            break;
```

```
        case MENU::CHECK:
```



```

        cout << a.getOwner() << "의 잔액은 " << a.check() << endl;
    }
} while (true);
return 0;
}

```

---

[소스]

//account.h

```
#ifndef ACCOUNT_H
```

```
#define ACCOUNT_H
```

```
#include <string>
```

```
using std::string;
```

```
enum class MENU{
```

```
    DEPOSIT = 1,
```

```
    WITHDRAW,
```

```
    CHECK,
```

```
    QUIT
```

```
};
```

```
class Account{
```

```
    string name;
```

```
    int balance;
```

```
public:
```

```
    Account(string n, int bal);
```

```
    ~Account();
```

```
    void deposit(int money);
```

```
    int withdraw(int money);
```

```
    int check() const;
```

```
    string getOwner() const;
```

```
};
```

```
#endif
```

//account.cpp

```
#include <string>
```

```
#include <iostream>
```

```
#include "account.h"
```

---

---

```
using namespace std;

Account::Account(string n, int bal) : name(n), balance(bal) { }

Account::~~Account() {
    cout << name << ": 객체 소멸" << endl;
}

void Account::deposit(const int money) {
    balance += money;
}

int Account::withdraw(int money) {
    if (money > balance) { // 저축된 돈이 찾는 금액보다 작은 경우
        money = balance;
        balance = 0;
    }
    else
        balance -= money;

    return money; // 실제 찾은 금액 리턴
}

int Account::check() const {
    return balance;
}

string Account::getOwner() const {
    return name;
}
}
```

---

[실행결과]

---